



Code-Signing Tool HSM

Quick Start Guide

Rev. 3.0.1

May 2018



Revision Sheet

Release No.	Date	Revision Description
Rev. 0	29/05/2018	Initial Work

USER'S GUIDE

TABLE OF CONTENTS

	<u>Page #</u>
A. Install	A-1
B. Configure	B-1
2.1 Install p11tool.....	B-1
2.2 Locate Token URL	B-2
2.2 Pull the public certificates.....	B-2
2.3 Binding.....	B-3
C. Run.....	C-4

**1.0 CST-HSM
Install
Configure
Run**

This document provides the information necessary for the user to effectively use Code-Signing Tool with Hardware Security Module backend. It is primarily intended for users who are familiar with CST tool to sign codes for the NXP High Assurance Boot (HAB). This guide assumes that the HSM is installed, configured and that all cryptographic materials are stored on it.

A.

The CST-HSM backend depends on:

- OpenSSL 1.0.2x (required by the Frontend)
- Libconfig

Make is required for building the software.

1) Install OpenSSL

```
# o ,fd hm kk nodm k kha k,cd
```

libconfig is C/C++ library for processing structured configuration files. It is used by CST to load HSM related configuration.

2) Install libconfig

```
# o ,fd hm kk khabnmehf,cd
```

Pre-compiled CST and HSM backend can be found on the delivered package.

3) Unzip the CST-HSM package.

```
# m ho b ,g l,2-/-0- ho
```

For Linux 32-bits cst binary can be found under cst-hsm-3.0.1/release/linux32/bin

For Linux 64-bits cst binary can be found under cst-hsm-3.0.1/release/linux64/bin

B.

CST requires having certificates on file system. To fulfill the requirement, you should pull the certificates used for signing to your file system using a utility which manage and use PKCS #11 security tokens. You can use the tool provided by your HSM vendor or any other alternative. In this manual we use p11tool.

2.1 Install p11tool

To perform interact with HSM you can use p11tool.

```
# o ,fd hm kk fm k ,ahm
```

2.2 Locate Token URL

- 1) Make sure that a token is initialized on your HSM.
- 2) Locate the PKCS #11 interface implementation library provided by your HSM vendor. It will be used for p11tool and later by CST.
(e.g)
Utimaco library is *libcs_pkcs11_R2.so*
SoftHSM2 library is *libsofthsm2.so*
- 3) Find the Token URL to interact with the token.

```
# o00 nnk ,,o n hcd ;obj 00 o n hcd kha o g= ,,kh , njdm
```

```
Snjdm /9
T 9
obj 009lncdk<Rne GRL$1/ 1:l m e b d <Rne GRL$1/onidb : d h k<8b0 da//d/4 2
37a: njdm< RS,GRL
adk9 RS,GRL
S od9 Fdmd hb njdm
L m e b d 9 Rne GRL o nidb
Lncdk9 Rne GRL 1
Rd h k9 8b0 da//d/4 237a
Lnc kd9 m kk(
```

In this case Token URL is

pkcs11:model=SoftHSM%20v2;manufacturer=SoftHSM%20project;serial=9c1aeb00e05a348b;token=CS
T-HSM

2.2 Pull the public certificates

Pull all certificates referenced on your Command Sequence File. Assume that a CSF and an IMG certificates were referenced.

- 1) List all certificates on HSM

Locate certificate URL

```
# o00 nnk ,,o n hcd ;obj 00 o n hcd kha o g= ; njdm, k= ,,kh , kk,
bd
```

- 2) Pull CSF certificate from HSM

Export the certificate to file system using its URL.

```
# o00 nnk ,,o n hcd ;obj 00 o n hcd kha o g= ,,d on ;bd k= ,,
n ehkd R 0 0 g 145 1/37 54426 2 b b -odl
```

3) Pull IMG certificate from HSM

Export the certificate to file system using its URL.

```
# o00 nnk ,,o n hcd ;objb 00 o n hcd kha o g= ,,d on ;bd k= ,,
n ehkd HLF0 0 g 145 1/37 54426 2 b b -odl
```

2.3 Binding

In the same folder as CST, create a **hsm.cfg** file. This file is used by CST to determine parameters to access HSM and to locate cryptographic objects needed for signing.

The configuration file specifies the following parameters.

- **hsm.module**
- **hsm.pin** specifies the User PIN, for security reason you may not specify this value, it will be asked at runtime.
- **hsm.slot** specifies the SLOT ID
- **hsm.objects** is a lookup table which bind a cryptographic material on the file system to its Object ID on the HSM. This is needed since the current frontend implementation loads and checks certificates from the file system and then provide as argument the certificate path to the backend. The backend can see a path only. To be load the required certificate and its private key from the HSM, the backend uses this lookup.

n **hsm.objects.file** path to certificate on file system.

n **hsm.objects.id** Identifier of the same certificate on the HSM.

Please note that this Id will be used also to load the private key corresponding to the certificate. HSM binds certificates to private keys by giving the same Id for both objects and in some HSM implementation they should have the same Label also.

```
# m mn g l -bef
```

```
g l ee
g l 9

lnc kd < ;objb 00 o n hcd kha o g= :
ohm < 012345 :
kn < /:
naidb <
    ehkd < --. --.b . R 0 0 g 145 1/37 54426 2 b -odl :
    hc < db6/4/07d8ae7 c5//85d02ba1e/ea c :

    ehkd < --. --.b . HLF0 0 g 145 1/37 54426 2 b -odl :
    hc < /b7b b/2874ea5cbdc18b86cb72 de6 :
( :
:
```


C.

Finally invoke the CST.

```
# -. b      , h ; hme hkd= , n ; n  ehkd=  
R      nbd  dc    bbd  e kk    mc  hfmdc c      hk akd hm ; n  ehkd=
```